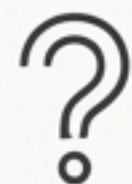


多Agent应用最佳实践

从概念到代码，构建你的第一个AI开发团队

AI技术专家分享

本次分享大纲：一场AI团队的软件开发之旅



1. 问题的提出：为何需要“AI团队”？

单个LLM在复杂任务中的局限性



2. 解决方案：认识我们的AI开发团队

多Agent协作模式的兴起



3. 项目蓝图：技术栈与代码结构解析

CrewAI框架及项目文件概览



4. 实战演示：看AI团队如何从零到一开发应用

需求分析到代码生成全过程



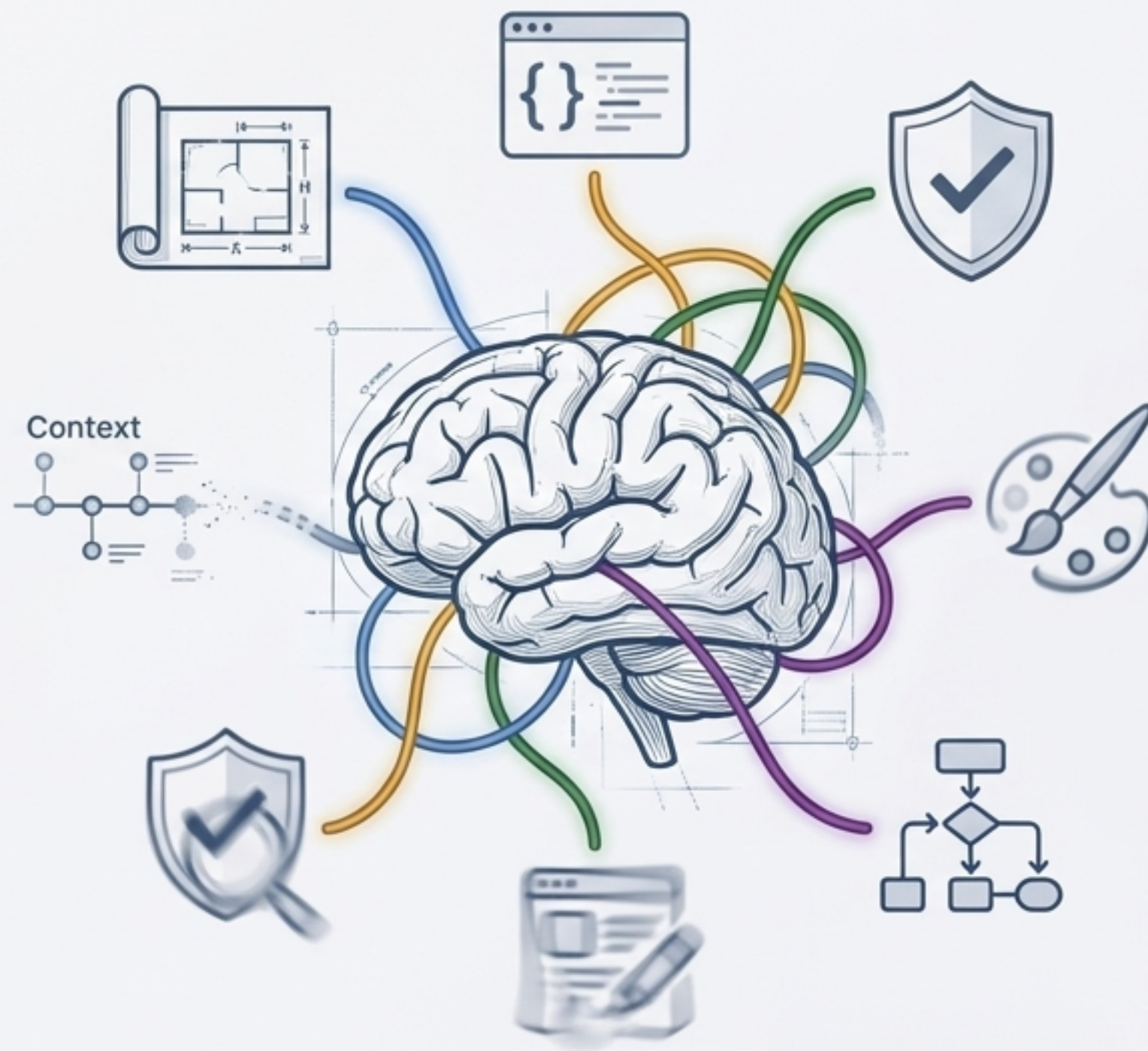
5. 复盘总结：关键实践与未来展望

提炼核心经验与最佳实践

挑战：当单个“超级大脑”独木难支

单个LLM的固有局限性

- 任务分解能力有限：难以将一个模糊的宏大目标拆解为具体、可执行的步骤。
- 缺乏专业化和深度：“通才”模型在特定领域（如系统架构设计、代码优化）的深度和经验不足。
- 上下文丢失与一致性问题：在长流程任务中，容易忘记早期决策，导致后续产出不一致。
- 验证和反思机制缺失：难以自我批判、审查和迭代自己的工作成果。



破局：组建一个各司其职的AI专家团队

将复杂问题拆解，分配给拥有不同角色、技能和工具的自主Agent，通过协作完成共同目标。



****专家洞察**:**

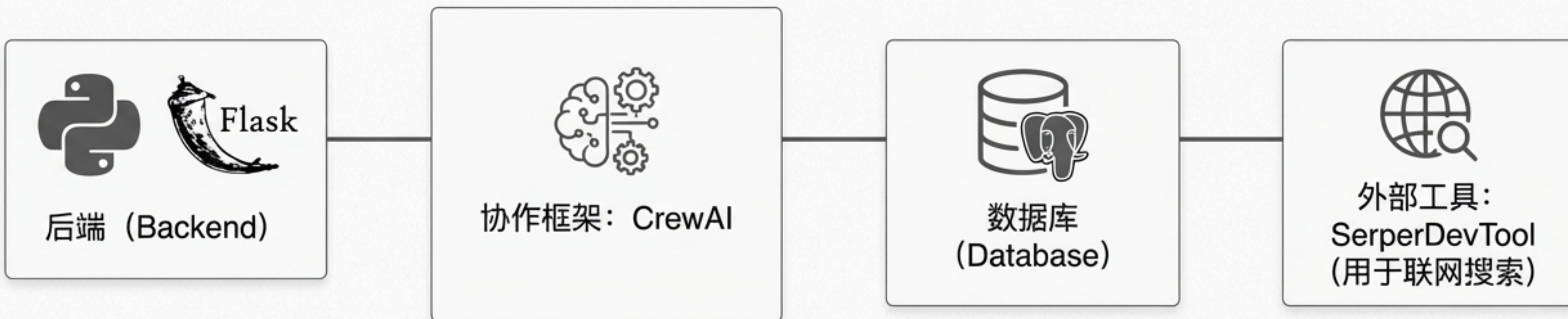
我们不再是与一个AI对话，而是成为一个AI团队的‘技术主管(Tech Lead)’。

本次任务：开发一个Python员工管理Web应用

用户需求

“我需要一个基于Python和Flask的简单Web服务，用于管理员工信息。功能包括员工的增删改查。”

项目关键技术栈



项目代码结构：一个多Agent应用的蓝图



认识团队成员：定义Agent的角色、目标与背景



需求分析师 (Requirement Analyst)

角色：负责分析用户需求，将其转化为结构化的技术规格。

```
requirement_analyst = Agent(  
    role='需求分析师',  
    goal='细致分析用户需求，确保完整、清晰、可行',  
    backstory='你是一位经验丰富的需求分析师...',  
    tools=[search_tool],  
    allow_delegation=True  
)
```



框架设计师 (Framework Designer)

角色：根据需求文档，设计健壮且可扩展的系统架构。

```
framework_designer = Agent(  
    role='框架设计师',  
    goal='设计一个项目所需的技术框架',  
    backstory='你是一位资深的架构师...',  
    tools=[search_tool],  
    allow_delegation=True  
)
```

“backstory”不仅仅是装饰，它为LLM提供了行为和沟通风格的上下文，使其角色扮演更具说服力。

认识团队成员：从代码生成到质量审核



代码生成器 (Code Generator)

角色：根据架构设计，编写高质量、可维护的代码。

```
code_generator = Agent(  
    role='代码生成',  
    goal='按照设计生成高质量代码',  
    backstory='你是一位注重代码质量和可维护性的程序员...',  
    allow_delegation=False  
)
```

注意：此Agent不允许委派任务



双重审核 (The Dual Reviewers)

合理性审核师：审查设计方案的合理性和可行性。

代码审核师：审查代码的质量、风格和潜在错误。

```
reasonableness_reviewer = Agent(role='合理性审核', goal='审查  
code_reviewer = Agent(role='代码审核', goal='审查生成的代码，
```


规划 workflow：将目标拆解为具体任务

定义任务 (Defining Tasks)

```
# 任务1: 收集和分析需求
collect_analyze_requirements = Task(
    description='分析用户对Python Web应用的需求, 输出详细需求文档',
    expected_output='一份详细的需求文档',
    agent=requirement_analyst # <-- 将任务分配给特定Agent
)

# 任务2: 设计系统架构
design_system_architecture = Task(
    description='根据需求文档设计系统技术框架',
    expected_output='一份详细的系统设计文档',
    agent=framework_designer # <--
)

...
```

组建团队并设定流程 (Assembling the Crew & Setting the Process)

```
# 创建Crew, 注入Agents和Tasks
crew = Crew(
    agents=[requirement_analyst, ...],
    tasks=[collect_analyze_requirements, ...],
    process=Process.sequential, # <-- 设定为顺序执行流程
    verbose=True
)

# 启动任务!
result = crew.kickoff()
```



Analyst

需求文档



Architect

设计文档



Coder

代码



Reviewer

执行阶段（1/4）：需求分析师的深度调研

Agent思考与行动

Thought: "我需要搜索关于构建Python Web应用的行业最佳实践，以便更好地理解用户需求。"

Action: Using Tool: Search the Internet

Tool Input: {"query": "Python web application requirements best practices"}

Tool Output:

Title: 8 Python Best Practices Every Developer Should Know

- Snippet: A comprehensive guide covering essential best practices for writing clean, efficient Python code, including web application structure and deployment.

Title: An ultimate guide to web development in Python

- Snippet: Explores popular Python web frameworks, design patterns, and key requirements for building scalable web applications.

Title: Python Web Development: Requirements and Best Practices

- Snippet: Details functional and non-functional requirements, security considerations, and best practices for user authentication and data handling in Python web projects.

... (and a few more results)

输出：专业需求文档（PRD）

项目需求文档

1. 用户认证和登录

- 支持邮箱和密码登录。
- 提供用户注册、密码重置功能。
- 实施基于OAuth的第三方登录集成。

2. 管理功能

- 用户管理界面，包括权限分配和角色设置。
- 数据仪表盘，展示关键业务指标。

3. 安全性

- 实施严格的数据加密和身份验证机制。
- 防止常见的Web攻击（如SQL注入、XSS）。

4. 界面设计

- 响应式Web设计，适配桌面和移动设备。
- 清晰导航和直观的用户体验。

技术要求

- 基于Python的Web框架（如Django或Flask）。
- 使用PostgreSQL或MySQL作为数据库。
- 部署在云服务平台（如AWS或GCP）。

Agent通过外部工具获取实时信息，将模糊需求具体化为一份专业的PRD文档。

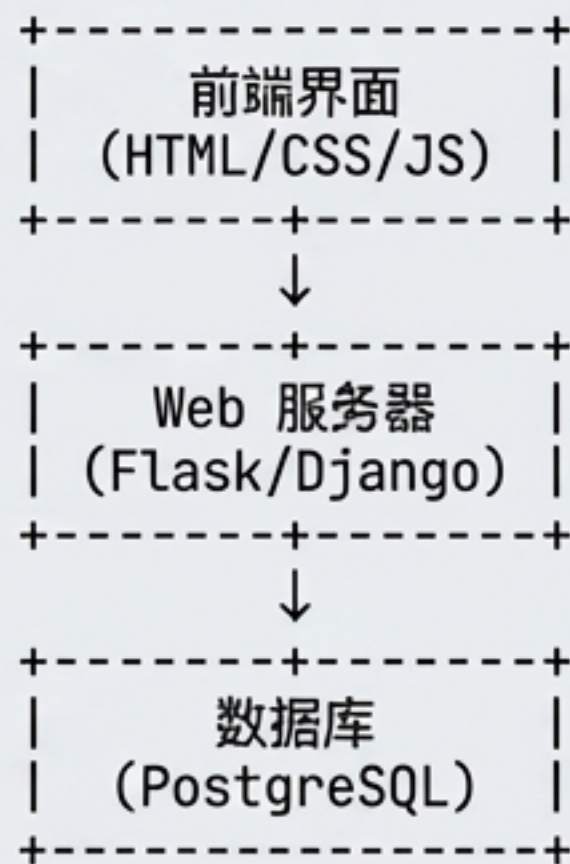
执行阶段（2/4）：架构师绘制系统蓝图



输入：需求分析师生成的详细需求文档。

"基于需求，我需要设计一个包含前端、Web服务器和数据库的三层架构。Flask是合适的Web框架，PostgreSQL作为数据库。"

输出：技术设计文档



Agent将文字需求转化为结构化的技术方案和可视化图表，为后续编码奠定基础。

执行阶段（3/4）：代码生成器构建应用骨架

 输入：框架设计师输出的系统设计文档。

输出：文件结构

```

└─ employee_management/
    ├── app.py # 主应用入口
    ├── models.py # 数据库模型
    ├── routes.py # 路由定义
    └─ templates/ # HTML模板
        └─ ...
    └─ static/ # 静态文件
        └─ ...
```

输出：核心代码

```
""" app.py: Flask应用主文件 """
from flask import Flask, render_template
from models import db, Employee
from routes import auth, employee

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'postgresql://user:pass@host/db'
db.init_app(app)

@app.route('/')
def home():
    return render_template('login.html')

if __name__ == '__main__':
    app.run(debug=True)
```

Agent不仅生成了代码，还创建了符合工程实践的、结构清晰的项目目录，可以直接运行。

执行阶段（4/4）：审核Agent的“代码评审”



输入：代码生成器生成的完整代码。

输出：代码审核报告

✓ **结论：**代码审核结论是合理的。

优点

- 可读性：代码结构清晰，遵循了PEP8规范。
- 设计合理性：模块划分清晰，符合设计文档。

改进建议

- 增加单元测试：建议为核心业务逻辑添加单元测试。
- 配置管理：将配置信息外部化，不要硬编码。
- 异常处理：增强数据库操作的异常处理。

最终交付物：一个完整且经过评审的应用程序

```
# Final Answer:  
import logging
```

```
class DataProcessor:   
    """数据处理类，负责处理原始数据"""
```

1. 代码实现了一个CRUD数据处理类`DataProcessor`...

```
    def __init__(self, data):  
        self.data = data
```

```
    def process_data(self):   
        # ... (some processing logic)  
        processed_data = [item for item in self.data]  
        return processed_data
```

2. `process\_data`方法是核心...

```
def main():   
    """主程序入口，调用数据处理"""  
    logging.basicConfig(level=logging.INFO)  
    data = ["apple", "banana", "cherry"]
```

3. 通过主函数`main`入口调用...

```
    processor = DataProcessor(data)  
    result = processor.process_data()  
    logging.info(f"处理结果: {result}")
```

```
if __name__ == "__main__":  
    main()
```

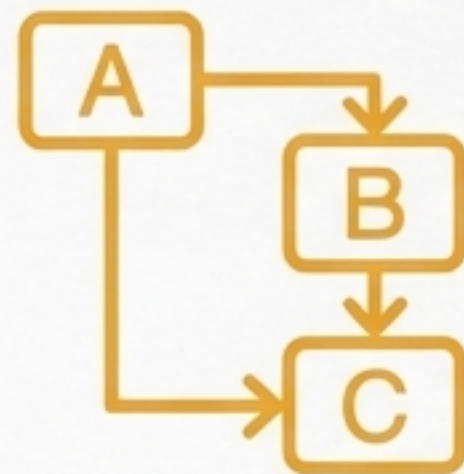
AI团队的产出不仅是代码，而是包含了代码、文档、测试建议的完整工程交付物。

核心实践总结：如何领导你的AI团队



1. 角色定义要“专” (Specialize Roles)

为每个Agent设定清晰、单一的职责。避免让一个Agent身兼数职。



2. 流程设计要“清” (Clarify the Process)

明确Agent间的协作模式（顺序、分层等）。顺序流程最适合有明确前后依赖的任务。



3. 工具授权要“准” (Provide a T-Shaped Toolset)

为Agent配备完成任务所必需的工具（如搜索、文件读写）。工具是能力的延伸。



4. 反馈闭环要“有” (Incorporate Feedback Loops)

引入审核、评审类Agent，形成验证和迭代的闭环，这是提升最终产出质量的关键。

多Agent系统不仅是自动化工具，更是一种全新的编程范式。思考一下，你的下一个项目，需要一支什么样的AI团队？